Proposal for System Hardening: Pointer-First Constitution and Operational Equilibrium

1. Architectural Integrity and Governance Analysis

The architecture of the Stela system relies fundamentally on the "Pointer-First" constitution, a governance model designed to minimize token consumption and maximize strict adherence to canonical truth. This model, enshrined in the *Policy of Truth* (PoT.md) [1], dictates that active work surfaces—specifically TASK.md—must function as routing logic and state containers rather than repositories of duplicated canonical text. The current operational state reveals specific friction points where the implementation diverges from this doctrine, leading to "Drift," data volatility during the closeout cycle, and unnecessary cognitive load on the agent due to context saturation.

The following analysis provides a forensic breakdown of the necessary hardening measures for TASK.md, the operational binaries (ops/bin/prune, tools/lint/style.sh), and the associated documentation (docs/MANUAL.md). This report serves as the definitive specification for the "Follow-up to TASK.md Overhaul" requested by the Operator, ensuring that every operational directive enforces the logic defined in PoT.md.[1]

1.1 The Pointer-First Imperative

The "Pointer-First" doctrine is not merely a stylistic choice; it is an operational imperative derived from the PoT.md requirement for "Reuse-First Discipline" and "Anti-Drift Governance".[1] The core axiom states that "SSOT: one canonical file per domain; other mentions are pointers".[1]

In the current instantiation of TASK.md, remnants of instructional text and redundant gate logic create a "dual-canon" hazard. If TASK.md contains a static copy of a rule defined in PoT.md, and PoT.md is updated, the static copy in TASK.md becomes stale immediately, creating a schism in the system's truth. The hardening proposed herein strips TASK.md of these redundancies, transforming it into a pure control plane interface that dynamically references PoT.md, SoP.md, and docs/MANUAL.md for behavioral instructions. This ensures that the agent is always operating on the most current version of the law, as accessed through the pointers in the llms.txt entry point.[1]

1.2 System Equilibrium and the Closeout Paradox

The PoT.md defines "Equilibrium Maintenance" as a state where "a task is considered

complete only when the system state reaches equilibrium and SoP.md is updated".[1] However, forensic analysis of the ops/bin/prune binary [1] and the docs/MANUAL.md closeout cycle [1] reveals a critical race condition—a "Closeout Paradox"—where the mechanism designed to restore hygiene (prune) actively destroys the evidence of equilibrium (the Work Log) before it can be permanently ledgered.

The operator has explicitly flagged this failure mode: "APPEARS THE WORKER DID NOT LEAVE THE TASK LOG ENTRY OR PRUNE WIPED IT OUT DURING THE SCRUB".[1] This report proposes a rigid "Safety Guard" implementation within the operational binaries to mechanically prevent this sequence failure, rather than relying on the "Contractor" (Guest AI) to remember the correct order of operations.

1.3 Context Economics and Signal-to-Noise Ratio

The effectiveness of the "Contractor" is inversely proportional to the noise in the context window. The current "Required Context Load" in TASK.md mandates the reading of dense bash scripts (tools/lint/dp.sh, ops/bin/prune).[1] Analysis suggests this is architecturally unsound. Unless the Dispatch Packet (DP) explicitly targets the refactoring of these tools, their source code provides no semantic value to the agent and consumes valuable token space that should be allocated to the PoT.md directives and the active code payload.

The proposal removes these artifacts from the mandatory load list, enforcing the principle that agents should verify the *output* of enforcement tools (via the receipts mechanism) rather than analyzing the *source code* of the tools themselves. This aligns with the "Black Box" verification model inherent in the OPEN-PORCELAIN and DUMP MANIFEST receipt requirements.

---

2. TASK.md Hardening: The Active Contract Refinement

The TASK.md file acts as the "Active Contract" and "Living Work Log".[1] It is the single surface where the Operator's intent is translated into Contractor action. The hardening measures detailed below address specific vulnerabilities in the Freshness Gate, Context Loading, and Receipt Verification protocols.

2.1 Section 3.1 Freshness Gate: Cryptographic Linkage

The Freshness Gate is the primary defense against "Stale State," a condition where an agent operates on a version of the repository that has been superseded. The current template allows for loose association between the Base HEAD and the artifacts provided.

Current Vulnerability:

The TASK.md template currently permits a disconnect where the Base HEAD listed in Section 1 (Session State) might differ from the artifacts listed in Section 3.1 if the Contractor fails to update them synchronously.

Proposed Hardening:

The refined schema requires explicit, cryptographic linkage. The Base HEAD hash must be the immutable key that binds all gate artifacts. The instructions must explicitly state that if the hash in the filename does not match the Base HEAD of the session, the gate is effectively closed.

Refined Specification for Section 3.1:

Field	Requirement	Validation Logic
Base HEAD	Must match Section 1 Base HEAD exactly.	grep comparison between Section 1 and 3.1.
OPEN Artifact	Must be the exact filename generated by ops/bin/open.	Filename must contain the Base HEAD hash.
OPEN-PORCELAIN	Must be the exact filename (or (none)).	Filename must contain the Base HEAD hash if present.
Dump Payload	Must be the exact filename generated by ops/bin/dump.	Filename must contain the Base HEAD hash.
Dump Manifest	Must be the exact filename generated by ops/bin/dump.	Filename must contain the Base HEAD hash.

The "Gate Commands" section is reaffirmed to include bash tools/lint/context.sh [1], ensuring that the CONTEXT.md manifest is respected before any work begins.

2.2 Section 3.2 Context Load: Optimization and Noise Reduction

The user query specifically challenges the inclusion of "NUMBERS 6 AND 7" in the "Required Context Load".[1] A review of the TASK.md file in the provided dump [1] identifies these items as:

6. tools/lint/dp.sh 7. ops/bin/prune

Forensic Justification for Removal:

- **tools/lint/dp.sh:** This is a 300+ line Bash script [1] responsible for regex-based validation of the TASK.md headings. While critical for the *system* to run, it is irrelevant for the *agent* to read. The agent's adherence to the TASK.md structure is governed by the template itself and the error messages returned by the linter, not by reading the linter's source code. Including it violates the "Pointer-First" principle of referencing high-level governance (PoT.md) rather than low-level implementation details.
- **ops/bin/prune:** This is a maintenance binary responsible for archiving SoP.md entries and cleaning storage/handoff.[1] Like the linter, the agent invokes this tool but does not need to understand its internal awk processing logic to execute a standard coding task.

Impact Analysis:

Removing these two files saves approximately 1,500-2,000 tokens of context (depending on tokenizer). This creates space for more relevant project documentation or code files, directly improving the agent's capacity to handle complex DPs without context fragmentation.

Revised Schema Block (Section 3.2):

The new list is strictly limited to the "Hard Constraints" defined in PoT.md:

1. PoT.md (Constitution)
2. SoP.md (History)
3. TASK.md (Active Surface)
4. docs/MAP.md (Terrain)
5. docs/MANUAL.md (Mechanics)

2.3 Section 3.4.5 Receipt: Provenance Enforcement

The operator explicitly requires: "PART OF #### 3.4.5 Receipt (Proofs to collect) NEEDS TO INCLUDE PRODUCTION OF OPEN OPEN-PORCELAIN DUMP (PLATFORM) AND DUMP MANIFEST (AS FILES FROM OPEN AND DUMP COMMAND NOT IN SUMMARY/RESULTS FILE)".[1]

Current Vulnerability:

Previous iterations of the TASK.md allowed agents to merely summarize that "State was captured." This effectively broke the chain of custody. If an error occurred, the Operator had no way to verify *exactly* what the agent saw at the start of the session.

Proposed Hardening: The "Receipt" section must be transformed from a free-text field into a mandatory checklist of specific file artifacts. This enforces "Quantitative Reporting" [1] by requiring the exact paths of the generated files.

Refined Specification for Section 3.4.5:

The section must explicitly list the following required proofs:

- **OPEN Artifact:** The filepath to storage/handoff/OPEN-...txt. This proves the session intent and git state.
- **OPEN-PORCELAIN:** The filepath to storage/handoff/OPEN-PORCELAIN-...txt. This proves whether the working tree was dirty (and exactly *how* dirty) at start.
- **Dump Payload:** The filepath to storage/dumps/dump-...txt. This proves the content available to the agent.
- **Dump Manifest:** The filepath to storage/dumps/dump-...manifest.txt. This proves the file list and exclusions applied.

This change ensures that every completed DP includes a full forensic snapshot of its starting conditions, enabling "Anti-Drift Governance".[1]

2.4 Section 4 Closing Block: The Results File Mandate

The operator states: "Mandatory Closing Block FILLED-IN FORM FIELDS NEED TO BE PROVIDED BY THE WORKER IN THE DP-OPS-00XX-RESULTS FILE".[1]

Current Vulnerability:

The TASK.md template currently places the "Mandatory Closing Block" (Commit Header, PR Title, etc.) at the bottom of the file. Agents frequently fill these fields *in place* inside TASK.md. When the prune --scrub command is run, TASK.md is reset, and this information is deleted.

Proposed Hardening:

The instructions in TASK.md Section 4 must be rewritten to explicitly forbid filling the fields in TASK.md. Instead, they must direct the worker to append these fields to the DP-OPS-XXXX-RESULTS.md artifact. This ensures the metadata survives the prune cycle and is available for the Operator during the PR merge process.

---

3. Operational Hygiene: The Prune Logic Fix

The operator reported a critical data loss incident: "APPEARS THE WORKER DID NOT LEAVE THE TASK LOG ENTRY OR PRUNE WIPED IT OUT DURING THE SCRUB".[1] This indicates a failure in the ops/bin/prune binary's handling of the TASK.md Work Log section.

3.1 Forensic Analysis of ops/bin/prune

Examination of the ops/bin/prune source code provided in the dump [1] reveals the root cause

in the scrub_task function:

Bash

```
# ops/bin/prune snippet
awk -v... '
  ...
  /^## 5\. Work Log \(Timestamped Continuity\)/ {
    print
    print "(No active thread)"
    exit
  }
  ...
' "${TASK_FILE}" > "${tmp_task}"
```

The Mechanism of Failure:

When ops/bin/prune --scrub is executed, the awk script scans TASK.md. Upon encountering the header ## 5. Work Log (Timestamped Continuity), it performs three actions:

1. Prints the header line.
2. Prints the placeholder string (No active thread).
3. **Exits immediately (exit).**

This logic aggressively truncates the file. If a Contractor has diligently written a detailed log entry describing their work *below* this header (as instructed by TASK.md), that entry is silently discarded. The exit command prevents awk from reading or preserving any subsequent lines.

3.2 The "Safety Guard" Proposal

To resolve this, the ops/bin/prune binary must be modified to implement a "Safety Guard." It must treat TASK.md Section 5 as a protected buffer that cannot be scrubbed if it contains unarchived data.

New Operational Logic:

1. **Pre-Scrub Check:** Before the scrubbing awk logic runs, the script must inspect Section 5 of TASK.md.
2. **Detection:** It must check for any text lines following the Section 5 header that do not match the standard placeholder (No active thread).
3. **Hard Stop:** If such lines are found, the script must abort with a fatal error: ERROR: Unarchived Work Log entries detected. Move to SoP.md before scrubbing.

This change shifts the responsibility from "Hope" (hoping the agent moved the log) to "Enforcement" (mechanically preventing the scrub until the log is moved). This aligns with the PoT.md directive: "The system prevents action that violates policy".[1]

---

4. Documentation: The Closeout Cycle Refinement

The docs/MANUAL.md file defines the "Closeout Cycle".[1] The current definition contributes to the data loss issue by being ambiguous about the *movement* of data.

4.1 Current Workflow Defect

The current text for Step 4 is:

4. Log
   Update SoP.md with a DP entry that includes verification receipts.

This instruction implies creating *new* content for SoP.md but does not explicitly instruct the removal or migration of the *existing* content in TASK.md. Consequently, agents leave the log in TASK.md (expecting it to persist) and then run Step 5 (prune), which deletes it.

4.2 The "Migration" Protocol

To support the logic fix in ops/bin/prune, the docs/MANUAL.md instructions must be updated to explicitly command a **cut-and-paste** operation.

Revised Specification for Closeout Cycle (Step 4):

4. Log
   MIGRATION (CRITICAL): Cut the session entry from TASK.md Section 5 and paste it into SoP.md as a new entry.
   VERIFICATION: Ensure TASK.md Section 5 is empty (containing only the header and placeholder) *before* proceeding to Prune.
   Update SoP.md with the DP entry, including objective summary and verification receipts.

This creates a closed loop: MANUAL.md tells the agent to clear the log, and the new ops/bin/prune binary refuses to run if they fail to do so.

---

5. Tooling Integrity: The Style Lint Fix

The operator reported: "AND THIS WARNING KEEPS COMING UP 'Install markdownlint if you

want style.sh to perform full checks' - I WANT IT FIXED".[1]

5.1 Forensic Analysis of tools/lint/style.sh

Examination of tools/lint/style.sh [1] reveals the following logic:

Bash

```
# tools/lint/style.sh snippet
if! command -v markdownlint >/dev/null 2>&1; then
  if]; then
    echo "ERROR: markdownlint is required..." >&2
    exit 1
  fi
  echo "WARN: markdownlint not found; skipping markdownlint checks." >&2
  skip_markdownlint=1
fi
```

The Root Cause: The script is designed to be permissive. It checks for the existence of markdownlint. If missing, it defaults to a warning (unless STRICT_MARKDOWNLINT is set). This permissiveness violates the PoT.md axiom of "Absolute Literalism".[1] If a script is named style.sh, its function is to enforce style. If it cannot perform that function due to missing dependencies, it is in a failure state, not a warning state.

5.2 The "Fatal Error" Proposal

The proposal converts this conditional warning into an unconditional fatal error. This forces the Operator (or the CI environment) to install the necessary dependency, ensuring that the "Linguistic Precision" standards are actually enforced rather than skipped.

Revised Logic Specification:

The script must immediately check for markdownlint. If absent, it must print instructions on how to install it and exit with status 1.

Bash

```
# Proposed Logic
```

```
if! command -v markdownlint >/dev/null 2>&1; then
    echo "ERROR: markdownlint is required. Install via 'npm install -g markdownlint-cli' or equivalent."
>&2
    exit 1
fi
```

This change eliminates the "alert fatigue" caused by the persistent warning and guarantees that a passing style.sh check actually means style compliance.

---

6. Detailed Implementation Manifest

The following table summarizes the specific modifications required to achieve the "Pointer-First" hardening and resolve the identified friction points. This serves as the execution plan for the Contractor.

6.1 Governance Surface Modifications

File	Section	Action	Rationale
TASK.md	**3.1 Freshness Gate**	**Hardening**	Enforce exact match between Session State Base HEAD and Gate Artifact filenames.
TASK.md	**3.2 Context Load**	**Pruning**	Remove entries #6 (dp.sh) and #7 (prune). Reduces token load; enforces output verification over code reading.
TASK.md	**3.4.5 Receipt**	**Expansion**	Explicitly list OPEN, OPEN-PORCELAIN, DUMP, and DUMP MANIFEST as mandatory checklist items.

TASK.md	**4. Closeout**	**Redirection**	Instruct worker to output "Mandatory Closing Block" fields to DP-OPS-XXXX-RESULTS.md, not TASK.md.
docs/MANUAL.md	**Closeout Cycle (Step 4)**	**Refinement**	Explicitly instruct the *migration* (Cut/Paste) of the Work Log from TASK.md to SoP.md to prevent data loss.

6.2 Binary Logic Modifications

File	Logic Block	Action	Rationale
ops/bin/prune	scrub_task function	**Safety Guard**	Add pre-flight check: Scan TASK.md Section 5. If unarchived content exists, abort scrub with Fatal Error.
tools/lint/style.sh	Dependency Check	**Strict Mode**	Remove WARN logic for missing markdownlint. Make absence of the binary a Fatal Error (exit 1).

---

7. Operational Implications and Token Economics

The shift to a stricter "Pointer-First" constitution has measurable benefits for the system's

operational capability.

7.1 Token Efficiency

By removing tools/lint/dp.sh and ops/bin/prune from the mandatory context load, we reclaim approximately 1,500 tokens per request.[1] In a system using "Deep Research" or complex reasoning models, these tokens are better allocated to the "Working Tree" context—the actual files being modified. This increases the model's ability to "reason" about the code changes rather than the enforcement machinery.

7.2 Auditability

The enforcement of OPEN-PORCELAIN and DUMP MANIFEST in the receipt block creates a cryptographic chain of custody.

- **OPEN-PORCELAIN** proves the exact state of the repo *before* the AI touched it (specifically, whether it was dirty).
- **DUMP MANIFEST** proves exactly which files were visible to the AI.
  This eliminates "plausible deniability" regarding hallucinations. If an agent claims a file didn't exist, the Manifest proves otherwise.

7.3 Data Persistence

The interplay between the docs/MANUAL.md update (instructing migration) and the ops/bin/prune update (enforcing migration) creates a "Poka-Yoke" (mistake-proofing) mechanism. The system will physically refuse to destroy the session history until it has been confirmed moved to the persistent ledger (SoP.md). This resolves the "worker did not leave the task log" error by making it impossible to finalize the task without doing so.

---

8. Conclusion

The proposed changes represent a "Hardening" phase for the Stela system. They transition the repository from a state of "suggested compliance" to "enforced constraints." By embedding the "Pointer-First" logic directly into the operational binaries and the active contract (TASK.md), we ensure that the "Behavioral Logic Standard" defined in PoT.md is not just a policy, but a physical law of the repository environment.

The execution of these changes will result in:

1. **Zero-Drift Task Execution:** Guaranteed by the Freshness Gate linkage.
2. **Lossless History:** Guaranteed by the prune Safety Guard.
3. **High-Fidelity Context:** Guaranteed by the optimized Context Load.
4. **Deterministic Verification:** Guaranteed by the style.sh Fatal Error fix.

This proposal satisfies all requirements set forth in the user query and establishes a robust foundation for future "Task System" scalability.

Works cited

1. dump-full-main-e46e200e.txt